claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a26929d4c969fb9c5.jsonl`
- SHA-256: `b216cff919720a0c447f009db8d2c02a72d391808da1aafc29dc8e9c2a214d49`
- Source modified: `2026-07-08T06:15:45+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T06:12:31.310Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to
an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).
The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):
- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical
  to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the
  SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config
  + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed
  once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing,
  never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #2.

CLAIM (medium/robustness) backend/orchestration.py:689

summary: When validation is delegated to the L4 worker
(phase_placement.validation=cloud_run_l4), a worker validation REJECTION surfaces to the user as
an opaque "cloud job failed (returncode 2)." with no reason, silently dropping the validation
feedback the in-process path provides.
failure scenario: Config: deployment.phase_placement.validation=cloud_run_l4 with
segment=cloud_run_l4 (or compute_target=cloud_run). A user uploads a too-blurry / not-badminton
clip. _execute_processing sets validate_on_control_plane=False (orch:865-868), so the control-
plane skips its validators and dispatches with skip_validation=worker_skips_validation=False
(orch:666-674). The worker's blur/relevance validator rejects the clip: cmd_infer hits `if
failures: return _fail(2)` (worker/__main__.py:333-338), which writes a done-marker via
_write_done_marker containing ONLY {video_id, returncode:2, segment_count:0, status:"failed"} ?
NO message/validator-reason field (worker/__main__.py:228-238). run_infer returns
ComputeResult(returncode=2); _maybe_dispatch_remote line 688-689 then returns _failed(f"cloud
job failed (returncode {result.returncode}).") ? it never reads result.report.get('message')
(unlike the compile path at orch:1393). The user sees "cloud job failed (returncode 2)." instead
of the specific 'video too blurry / not badminton' message the in-process control-plane path
returns. This failure mode is newly reachable in #521 (pre-#521 skip_validation was hardcoded
True so the worker never validated) and is untested ? the new tests
(test_validation_delegated_to_worker_when_placed_on_l4 / _not_delegated_...) only cover the PASS
path and never assert the worker-rejection message.

2. user (2026-07-08T06:12:40.526Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
---BRANCH---
codex/issue-523-sd-reel-plan

3. user (2026-07-08T06:12:41.972Z)

dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
8866ffe perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521)
428ee44 address follow-up review on #169: correct factual error in item-3 investigation note
(fusion_compare.py, fusion_golden.py, ablation.py + tests *do* exist per ls/grep on the tree;
person features in fusion_hybrid + detectors/fusion_features.py; remaining M0 in-container task
is wiring into compare). Cleaned duplicate sequencing paragraph. Verified files present. Plan
now accurate. M0 item 3 placement remains correct and even stronger.
318f6c2 test: positive tests for Item 2 (config) - shallow section replacement, unknown keys
warning, get shim, roundtrip + living doc update for T2 post #157
8836dfe chore: scripts/ for run-drivers + CODE_MAP §0 file-placement guidelines (#138)
1358521 review: address #42 nits ? dynamic section scan, shim comment, scalar-default test
5257fd5 feat: pipelined YOLO chunk processing & unified Gemini model config

4. user (2026-07-08T06:12:51.984Z)

1460 /c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/orchestration_525.py

5. user (2026-07-08T06:12:54.556Z)

977 /c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer/75d3d121-f240-4293-afa5-b436a379ad31/scratchpad/worker_main_525.py

6. user (2026-07-08T06:13:03.131Z)

```
620                 False,
621             )
622
623         notify("dispatching (cloud_run)")
624         logger.info(
625             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
626             req.video_path,
627             out_root,
628         )
629         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
630         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
631         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
632         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
633         overrides = []
634         idx_cfg = getattr(cfg, "indexing", None)
635         sk = getattr(idx_cfg, "skip_ai_handoff", None)
636         if sk is not None:
637             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
638         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
639         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
640         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
641         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
642         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
643         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
644         wasb_cfg = getattr(idx_cfg, "wasb", None)
645         if wasb_cfg is not None:
646             decode_backend = getattr(wasb_cfg, "decode_backend", None)
647             if decode_backend:
648                 overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
649             batch_size = getattr(wasb_cfg, "batch_size", None)
650             if batch_size:
651                 overrides.append(f"indexing.wasb.batch_size={batch_size}")
652             prefetch = getattr(wasb_cfg, "prefetch_batches", None)
653             if prefetch:
654                 overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
655         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
656         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
657         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
658         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
659         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
660         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
661         # Who validates? phase_placement.validation (#521). DEFAULT (control_plane): the
control-plane is
662         # the authoritative gate ? it already validated with its resolved config (only a
PASS reaches
663         # here), so the worker SKIPS its redundant re-validation (#513). If validation is
placed on the
664         # L4 (cloud_run_l4), the control-plane delegated it (it did NOT run the validators ?
```

```
see
665         # _execute_processing), so the worker must NOT skip: it becomes the gate.
666         worker_skips_validation = (
667             resolve_phase_placement(cfg, PHASE_VALIDATION) != PLACE_CLOUD_L4
668         )
669         spec = ComputeJobSpec(
670             video_uri=req.video_path,
671             out_uri=out_root,
672             segmenter=seg_name,
673             config_overrides=tuple(overrides),
674             skip_validation=worker_skips_validation,
675         )
676         try:
677             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
678             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
679             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
680             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
681             result = compute.run_infer(
682                 spec,
683                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
684             )
685         except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
686             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
687             return _failed(f"cloud dispatch error: {e}", True)
688         if result.returncode != 0:
689             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
690         if result.video_id and result.video_id != video_id:
691             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
692             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
693             logger.warning(
694                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
695                 "and the worker hash; ingesting under the local id.",
696                 result.video_id,
697                 video_id,
698             )
699
700         notify("ingesting (cloud_run)")
701         storage_dict = (
702             storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy
can't narrow
703             if hasattr(storage_cfg, "model_dump")
704             else {}
705         )
706         try:
707             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
708         except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
709             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
710             return _failed(
711                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
712                 True,
713             )
714
715         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
716         segments = [
717             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
718         ]
719         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
```

```
720         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
721         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
722         provenance = {
723             "path": "cloud_run",
724             "requested": req.compute,
725             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
726             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
727             "execution": result.execution,
728             "outputs_uri": result.outputs_uri,
729         }
730         logger.info(
731             "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
732             result.execution,
733             provenance["job"],
734             provenance["region"],
735             result.outputs_uri,
736             video_id,
737         )
738         return (
739             {
```

7. user (2026-07-08T06:13:04.095Z)

```
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
214         return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
228         try:
229             marker = {
230                 "video_id": video_id,
231                 "returncode": returncode,
232                 "segment_count": segment_count,
233                 "status": status,
234             }
235             local = os.path.join(scratch, "_done.json")
236             with open(local, "w", encoding="utf-8") as f:
```

```
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote completion marker -> %s", done_uri)
        except Exception as e:  # never fail a good run because the marker push hiccuped
            logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)


    def cmd_infer(args: argparse.Namespace) -> int:
        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        if not _run_startup_checks(config):
            return 2

        # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
        # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
        # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
        # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
        compute = get_compute_backend(config)
        if compute is not None:
            return _dispatch_remote(compute, args)

        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
        os.makedirs(scratch, exist_ok=True)
        config.output.output_dir = scratch

        # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
        local_video = storage.fetch(
            args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
        )
        print(f"[worker] pulled {args.video_uri} -> {local_video}")

        # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
        video_id = compute_video_id(local_video)

        # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
        # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
        # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
        # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
        # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
        # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
        # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
        # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
        # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
        if getattr(args, "skip_validation", False):
            print(
                "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
                "gate and already validated this input."
            )
```

```
            val_results, val_context = [], {}
        else:
            val_results, val_context = validator_registry.run_all_with_context(
                local_video, config
            )
        failures = [r for r in val_results if not r.passed]
        db = Database(os.path.join(scratch, config.output.db_name))
        db.add_video(
            video_id,
            local_video,
            "failed" if failures else "processed",
            [r.model_dump() for r in val_results],
        )

        def _fail(rc: int) -> int:
            # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
            # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
            # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
            # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
            # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
            # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
            # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
            try:
                _serialize_and_push_outputs(
                    scratch,
                    args.out_uri,
                    video_id,
                    [],
                    "failed",
                    storage_cfg,
                    str(getattr(args, "video_uri", "") or ""),
                )
            except Exception as e:  # never mask the real failure on an artifact-push hiccup
                logger.warning("[worker] could not push failure artifacts: %s", e)
            # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
            if getattr(args, "done_uri", None):
                _write_done_marker(
                    args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
                )
            return rc

        segments: List[dict] = []
        segmenter_actual = None
        segmentation_ran = False
        status = "failed"
        try:
            if failures:
                print(
                    "[worker] validation FAILED: "
                    + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
                )
                return _fail(2)
            seg_name = args.segmenter or config.indexing.default_segmenter
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(
                    f"[worker] unknown segmenter: {seg_name!r} "
                    f"(have {list(segmenter_registry.list_available())})"
```

```
345                 )
346                 return _fail(2)
347             segmenter_actual = seg_name
348             result = segmenter_cls(db, config).process_video(
349                 video_id, local_video, max_frames=args.max_frames
350             )
351             segmentation_ran = True
352             if isinstance(result, Failure):
353                 print(f"[worker] segmenter FAILED: {result.message}")
354                 return _fail(3)
355             segments = result.value if hasattr(result, "value") else result
356             status = "success"
357             # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358             # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359             # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
```

8. user (2026-07-08T06:13:28.180Z)

```
840                     )
841                     return response
842
843             # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
844             #    UNLESS this exact content already has a cached verdict under the current
validator config.
845             notify("validating")
846             # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
847             # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
848             # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
849             # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
850             # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
851             # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
852             # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
853             # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
854             # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
855             # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
856             # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
857             #
858             # #521 phase_placement.validation: when validation is placed on the L4 AND
detection will
859             # actually dispatch there, the control-plane DELEGATES validation to the worker
? it does NOT
860             # run the validator suite here (the whole point is to move the CPU-bound
validation OFF the
861             # 2-vCPU control-plane). The worker then validates (skip_validation=False) and
rejects a bad
862             # clip via a non-zero rc, which _maybe_dispatch_remote surfaces as a failed job.
When detection
863             # is NOT on the cloud (in-process), the control-plane always validates ? there
is no worker to
864             # delegate to (the config guard makes validation=cloud_run_l4 require a cloud
```

```
segment).
            validate_on_control_plane = (
                resolve_phase_placement(config, PHASE_VALIDATION) != PLACE_CLOUD_L4
                or not _will_detect_on_cloud(config, getattr(req, "compute", None))
            )
            if not validate_on_control_plane:
                logger.info(
                    "[API] validation delegated to the L4 worker "
                    "(deployment.phase_placement.validation=cloud_run_l4) ? the control-
plane skips its "
                    "validator suite; the worker is the gate for %s.",
                    req.video_path,
                )
            val_fingerprint = (
                registry.config_fingerprint(config) if validate_on_control_plane else ""
            )
            # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
            # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
            # failing the job. The cache is a pure speedup ? never a new failure mode.
            reused_validation = False
            if validate_on_control_plane and not req.force:
                try:
                    cached = db.get_validation_cache(video_id)
                    if cached is not None and cached.get("fingerprint") == val_fingerprint:
                        # Replay the stored ValidationResult list; val_context stays {} (no
fresh
                        # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
                        # deliberate cost of skipping the decode). Segmentation reads
neither of these.
                        val_results = [
                            ValidationResult(**r) for r in cached.get("results", [])
                        ]
                        reused_validation = True
                        logger.info(
                            "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
                            "validator config unchanged ? skipping the validator suite. "
                            "force=true re-validates.",
                            cached.get("passed"),
                            video_id,
                        )
                except Exception:  # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
                    reused_validation = False
                    logger.warning(
                        "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
                        video_id,
                        exc_info=True,
                    )
            if validate_on_control_plane and not reused_validation:
                logger.info(f"Running validations for {req.video_path}...")
                val_results, val_context = registry.run_all_with_context(local_video,
config)
            failures = [r for r in val_results if not r.passed]
            if validate_on_control_plane and not reused_validation:
                # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
                # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
                db.set_validation_cache(
                    video_id,
```

```
                    val_fingerprint,
                    not failures,
                    [r.model_dump() for r in val_results],
                )

            if failures:
                # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
                # status; it no longer destroys the video's prior good segments.
                db.add_video(
                    video_id,
                    req.video_path,
                    "failed",
                    [r.model_dump() for r in val_results],
```

9. user (2026-07-08T06:13:29.333Z)

```
        spec_payload = {
            "video_id": video_id,
            "video_uri": video["filepath"],
            "output_name": out_name,
            "locale": locale,
            "time_pairs": [
                [float(s["start_time"]), float(s["end_time"])] for s in kept
            ],
            "stitching": stitch_dump,
        }
        token = uuid.uuid4().hex
        spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
        try:
            notify("staging compile spec")
            with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
                local_spec = os.path.join(td, "spec.json")
                with open(local_spec, "w", encoding="utf-8") as f:
                    json.dump(spec_payload, f)
                storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
        except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
            logger.error(
                "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
            )
            return {
                "status": "failed",
                "message": f"cloud compile: could not stage the compile spec: {e}",
                "video_id": video_id,
            }

        # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
        # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
        overrides = [f"storage.output_root={out_root}"]
        backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
        if backend_name:
            overrides.append(f"storage.backend={backend_name}")
        spec = CompileJobSpec(
            out_uri=out_root,
            spec_uri=spec_uri,
            video_id=video_id,
            output_name=out_name,
            config_overrides=tuple(overrides),
        )

        notify("dispatching stitch (cloud_run)")
        logger.info(
```

```
            "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
            video_id,
            out_root,
            len(kept),
        )
        try:
            result = compute.run_compile(
                spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
            )
        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
            logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
            return {
                "status": "failed",
                "message": f"cloud compile dispatch error: {e}",
                "video_id": video_id,
            }
        if result.returncode != 0:
            msg = str(
                result.report.get("message")
                or f"cloud compile job failed (returncode {result.returncode})."
            )
            return {"status": "failed", "message": msg, "video_id": video_id}

        download_url = str(
            result.report.get("download_url")
            or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        )
        logger.info(
            "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
            result.execution,
            result.outputs_uri,
            video_id,
        )
        notify("finished")
        return {
            "status": "success",
            "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
            "download_url": download_url,
            "video_id": video_id,
            "compute": {
                "path": "cloud_run",
                "execution": result.execution,
                "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
                "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            },
        }


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
        the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
        result for an EXPECTED problem (missing video / no segments / a stitch error) so the
SPA always gets
        a clean verdict instead of a worker-crash error.

        #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
        dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
        (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
        2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
```

```
here, unchanged."""
1433        notify = progress or (lambda stage: None)
1434        params = json.loads(job.get("params") or "{}")
1435        video_id = job.get("video_id") or ""
1436        segment_ids = params.get("segment_ids") or []
1437        locale = params.get("locale")
1438        output_filename = params.get("output_filename", "")
1439
1440        if (
1441            resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1442            and _cloud_available(cfg)
1443        ):
1444            return _dispatch_compile_remote(
1445                cfg, db, video_id, segment_ids, output_filename, locale, notify
1446            )
1447
1448        notify("resolving")
1449        try:
```

10. user (2026-07-08T06:13:41.392Z)

```
922             if failures:
923                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
924                 # status; it no longer destroys the video's prior good segments.
925                 db.add_video(
926                     video_id,
927                     req.video_path,
928                     "failed",
929                     [r.model_dump() for r in val_results],
930                 )
931                 response = {
932                     "video_id": video_id,
933                     "status": "failed",
934                     "message": _validation_failure_message(failures),
935                     "results": [r.model_dump() for r in val_results],
936                 }
937                 return response
938
939             # 2. Run segmenter & indexer
940             logger.info("[API] Video passed checks. Segmenting and indexing...")
941             db.add_video(
942                 video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
943             )
944
945             segmenter_cls = segmenter_registry.get(seg_name)
946             if not segmenter_cls:
947                 # Submit-time validation already 400s unknown names; this is the defensive
948                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
949                 available = list(segmenter_registry.list_available().keys())
950                 response = {
951                     "video_id": video_id,
952                     "status": "failed",
953                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
954                     "results": [r.model_dump() for r in val_results],
955                     "play_segments": [],
956                 }
957                 return response
958
959             logger.info(f"[API] Using segmenter: {seg_name}")
960             notify(f"segmenting ({seg_name})")
961             segmenter_actual = seg_name
962             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
```

```
963             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
964             play_wm, cand_wm = db.segment_watermarks(video_id)
965             # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
966             # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
967             # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
968             # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
969             # they're dropped on the cloud path in this slice ? a documented follow-up.)
970             remote = _maybe_dispatch_remote(
971                 config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
972             )
973             if remote is not None:
974                 response, segmentation_ran = (
975                     remote  # `ran` = did the cloud job actually execute (report)
976                 )
```

11. user (2026-07-08T06:13:43.805Z)

```
472:    def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
```

12. user (2026-07-08T06:13:45.465Z)

```
14:(validator registry ? segmenter ? report), never a forked pipeline.
40:from backend.reporting import emit_indexer_report
126:    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and push
both to
130:    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
132:    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
141:        os.path.join(out_local, "report.json"),
217:def _write_done_marker(
304:        # a status="failed" report.json/intervals.csv (0 segments) so the control plane's
bucket
323:            _write_done_marker(
336:                + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
353:            print(f"[worker] segmenter FAILED: {result.message}")
380:        # Best-effort per-video observability report (never raises) ? parity with the main
CLI.
381:        emit_indexer_report(
416:        _write_done_marker(
444:    """#521: write the compile done-marker (rc + status + the reel's download_url/reel_uri)
so the
451:            "download_url": result.get("download_url", ""),
452:            "reel_uri": result.get("reel_uri", ""),
453:            "message": result.get("message", ""),
464:def cmd_compile(args: argparse.Namespace) -> int:
516:        print(f"[worker] compile FAILED: {result.get('message')}")
558:    inference report (it flags the fallback) but corrupting for REAL training data. So here
we
884:        help="storage URI for the compile completion MARKER (rc + download_url), so a remote
"
900:    pc.set_defaults(func=cmd_compile)
964:        format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
```

13. user (2026-07-08T06:14:57.722Z)

```
472         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
473             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
474             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
```

```
475             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
476             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
477             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
478             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
479             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
480             return (
481                 {
482                     "video_id": video_id,
483                     "status": "failed",
484                     "message": msg,
485                     "results": [r.model_dump() for r in val_results],
486                     "play_segments": [],
487                     "compute": {
488                         "path": "cloud_run" if ran else "not_run",
489                         "requested": req.compute,
490                         "target": "cloud_run",
491                         "dispatched": ran,
492                     },
493                 },
494                 ran,
495             )
496
497         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
498         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
499         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
500         choice = (req.compute or "").strip().lower()
501         if choice and choice not in ("local", "cloud"):
```

14. user (2026-07-08T06:14:58.824Z)

```
440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
443     ) -> None:
444         """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
445         dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
446         try:
447             marker = {
448                 "video_id": video_id,
449                 "returncode": returncode,
450                 "status": result.get("status", "failed"),
451                 "download_url": result.get("download_url", ""),
452                 "reel_uri": result.get("reel_uri", ""),
453                 "message": result.get("message", ""),
454             }
455             local = os.path.join(scratch, "_compile_done.json")
456             with open(local, "w", encoding="utf-8") as f:
457                 json.dump(marker, f)
458             storage.put(local, done_uri, config=storage_cfg)
459             logger.info("[worker] wrote compile marker -> %s", done_uri)
460         except Exception as e:  # never fail a good stitch because the marker push hiccuped
461             logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)
462
463
464     def cmd_compile(args: argparse.Namespace) -> int:
```

```
465         """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
466         2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
467         the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
468         the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
469         forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
```

15. assistant (2026-07-08T06:15:37.227Z)

I've traced the complete factual chain of the claim and confirmed every step, plus the surrounding design intent. Here is my assessment.

What I confirmed (the claim's facts are all accurate):
- `orchestration.py:865-868` ? when `phase_placement.validation=cloud_run_l4` and detection is on cloud, `validate_on_control_plane=False`, so the control-plane skips its validator suite (`registry.run_all_with_context` at 908-910 is guarded and does not run).
- `orchestration.py:666-668` ? `worker_skips_validation` becomes `False`, so `skip_validation=False` is dispatched; the worker becomes the gate.
- `worker/__main__.py:333-338` ? `_fail(2)` ? `_write_done_marker` (217-238) writes only `{video_id, returncode, segment_count, status}` ? no message/validator-reason field on the infer marker.
- `orchestration.py:688-689` returns `_failed(f"cloud job failed (returncode {result.returncode}).", True)`; the `_failed` helper (472-495) fills `results` from the control-plane's `val_results`, which is empty on the delegated path. So the user gets `{status: failed, message: "cloud job failed (returncode 2).", results: []}`.
- Contrast: in-process path (922-937) returns `_validation_failure_message(failures)` + full `results`; the compile path (1391-1396) reads `result.report.get("message")` because the compile marker (worker 447-453) carries a `message` field.

Why I nonetheless rule is_real=false: The described "failure" is a reduction in error-message specificity on the delegated path ? not a wrong result, crash, or true data loss. The clip is correctly rejected and surfaced as a clean `status="failed"` job (the design's core invariant for failures is upheld). This behavior is explicitly documented as intended at `orchestration.py:858-864` ("the worker then validates ... and rejects a bad clip via a non-zero rc, which `_maybe_dispatch_remote` surfaces as a failed job"), and it only occurs under an explicit, non-default opt-in config (default `control_plane` uses the in-process path and surfaces the full message). The infer done-marker never carried a validator message (pre-#521 schema from #515/M6), so there is no available-but-dropped data in a wiring sense ? it is a design boundary. The "missing test coverage" is of message content, not of a functional failure mode (the job does fail correctly). Under the task's strict rubric (wrong result / crash / silent data loss / missing coverage of a real failure mode; no style/robustness/speculative), this is a UX/robustness message-quality concern, not a reportable correctness defect.

16. user (2026-07-08T06:15:44.920Z)

Structured output provided successfully